Universidad ICESI

Estrategias de Branching, Pruebas y Calidad Continua

GitFlow vs TBD, JaCoCo, SonarQube y Orquestación en Jenkins y Nexus

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI — Período 2026-2

Ruta de Aprendizaje de la Sesión

1. **Estrategias de Branching y Gobernanza Git**
2. Pirámide y Automatización de Pruebas en el Build
3. Cobertura con JaCoCo y Análisis Estático con SonarQube
4. Orquestación CI/CD: Jenkins, Quality Gates y Nexus

Universidad icesi

01

Estrategias de Branching

De ramas de larga vida al flujo continuo de Trunk-Based Development

Branching: GitFlow vs Trunk-Based Development

GitFlow (Ramas Tradicionales)

Estructura pesada con múltiples ramas:

- Ramas de larga vida (`develop`, `release`).
- Aislamiento prolongado del trabajo individual.
- **Merge Hell:** Conflictos masivos al integrar.
- Entregas lentas por lotes (*Batch Releases*).

Inadecuado para ciclos de entrega continua ágiles.

Trunk-Based Development (TBD)

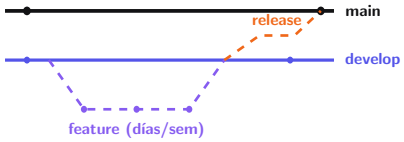
Flujo continuo sobre una sola rama principal:

- Integración diaria de cambios en `main`.
- Ramas efímeras (*Short-lived*): < 24–48 horas.
- Integraciones atómicas y resolución continua.
- Desacople vía **Feature Flags / Toggles**.

Optimiza métricas DORA clave (DF y LTTC).

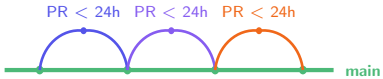
Topología Comparada de Ramificación

Modelo GitFlow (Complejidad Alta)



- Ramas divergentes acumulan deuda de merge.
- Ceremonias de estabilización costosas y manuales.
- Integraciones tardías con conflictos severos.

Trunk-Based Development (Flujo Continuo)



- Ramas de vida corta (< 24h) integradas a main.
- Cada merge ejecuta el pipeline de CI automáticamente.
- Código siempre listo para ser desplegado.

Métricas DORA: Rendimiento y Gobernanza DevOps

DORA (*Google Cloud / Accelerate*) define 4 métricas cardinales que balancean **velocidad** y **estabilidad**:

1. Frecuencia de Despliegue (DF)

Ritmo de entrega a producción:

- **Elite:** Múltiples veces al día (*On-Demand*).
- **Clave:** Trunk-Based y ramas efímeras (< 24h).

2. Tiempo de Entrega (LTTC)

Tiempo transcurrido de commit a producción:

- **Elite:** Menos de 1 hora (*Lead Time*).
- **Clave:** Pipeline CI/CD en Jenkins automatizado.

3. Tasa de Fallos en Cambios (CFR)

Porcentaje de despliegues con fallos:

- **Elite:** 0 % – 15 % de cambios defectuosos.
- **Clave:** Quality Gates con JaCoCo y SonarQube.

4. Tiempo de Recuperación (MTTR)

Tiempo de restauración ante incidentes:

- **Elite:** Menos de 1 hora de remediación.
- **Clave:** Artefactos inmutables en Nexus y Feature Flags.

Metas Ingesoft V: Maximizar velocidad (DF, LTTC) protegiendo la estabilidad operativa (CFR, MTTR).

Políticas de Pull Request y Code Reviews Efectivos

- **Tamaño atómico de PRs:** Cambios menores a 400 líneas. Las PRs pequeñas se revisan con rigor; las gigantes acumulan riesgo.
- **Descripción contextualizada:** Explicar el *por qué*, adjuntar evidencias de pruebas y vincular la tarea de Jira.
- **Foco en arquitectura y lógica:** Validar diseño, casos de borde y seguridad. Delegar formato y estilo a linters en el CI.
- **SLA de revisión ágil:** Responder en menos de 4 horas hábiles para evitar cuellos de botella en el flujo.

Cultura de Ingeniería

El Code Review **no** es una auditoría burocrática ni punitiva. Es una práctica colectiva de mentoría, alineación técnica y blindaje antes de llegar a producción.

Protección de Ramas: Blindando la Integridad de Main

Controles de Aprobación Humana

- **Bloqueo de push directo:** Nadie (incluyendo líderes o admins) comitea directamente sobre `main`.
- **Aprobaciones mínimas obligatorias:** Requerir al menos 1 o 2 aprobaciones de revisores pares cualificados.
- **Invalidación de aprobaciones:** Al subir nuevos commits a la PR, las aprobaciones previas se descartan automáticamente (*Dismiss stale reviews*).
- **Restricción de permisos destructivos:** Prohibir totalmente `git push --force` y la eliminación de ramas principales.

Status Checks Automatizados (CI)

- **Ejecución obligatoria del pipeline:** El merge queda deshabilitado a nivel de GitHub/Bitbucket hasta que Jenkins reporte estado verde.
- **Rama actualizada (Strict Branch):** Exigir que la rama de la PR esté sincronizada con la punta de `main` antes de fusionar.
- **Aprobación del Quality Gate:** SonarQube debe emitir dictamen aprobatorio para habilitar el botón de merge.
- **Estrategia de merge limpia:** Promover *Squash and Merge* o *Rebase and Merge* para preservar un historial lineal.

1. Estrategias de Branching y Gobernanza Git
2. **Pirámide y Automatización de Pruebas en el Build**
3. Cobertura con JaCoCo y Análisis Estático con SonarQube
4. Orquestación CI/CD: Jenkins, Quality Gates y Nexus

Universidad icesi

02

Pirámide y Automatización de Pruebas

Pruebas Unitarias, de Integración y Validación Continua en el Pipeline

La Pirámide de Pruebas y el Principio Shift-Left

- **Base (Unitarias):** Velocidad milimétrica, costo mínimo, aíslan la lógica del componente. Deben ser el 70–80 % de la suite.
- **Centro (Integración):** Validan interacción real entre capas (persistencia, serialización, contratos API). Ejecutan en segundos.
- **Cúspide (E2E / Sistema):** Flujos completos de usuario (Playwright). Lentas, frágiles y de mayor costo de mantenimiento.
- **Principio Shift-Left:** Detectar defectos en el commit local reduce el costo de corrección de 100x a 1x.

El Antipatrón del Cono de Helado

Invertir la pirámide (muchas pruebas manuales/E2E y pocas unitarias) satura los agentes de Jenkins, retrasa el feedback a horas y genera falsos positivos.

Pruebas Unitarias vs. Pruebas de Integración

Pruebas Unitarias (Unit Tests)

Aislamiento total y máxima velocidad:

- Verifican una única función, clase o regla de negocio.
- **Cero dependencias externas:** Sin acceso a bases de datos, red ni sistema de archivos.
- Uso intensivo de dobles de prueba con **Mockito** (`@Mock`, `@InjectMocks`).
- Ejecución automática con Maven Surefire (`mvn test`). Cientos de tests en 3–5 segundos.

Pruebas de Integración (IT)

Validación de interoperabilidad real:

- Verifican el contexto de la aplicación (`@SpringBootTest`, repositorios JPA, controladores REST).
- Uso de entornos efímeros reales mediante **Testcontainers** (PostgreSQL, Redis en Docker) o base en memoria.
- Detectan errores de mapeo SQL, transaccionalidad y contratos HTTP.
- Ejecución con Maven Failsafe (`mvn verify`).

Implementación: Unit Test (Mockito) vs Integration Test (Spring)

Prueba Unitaria: Aislamiento con Mockito

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
  @Mock
  private OrderRepository orderRepo;
  @InjectMocks
  private OrderService orderService;

  @Test
  void shouldCalculateDiscount() {
    when(orderRepo.findTier("A01"))
        .thenReturn(Tier.GOLD);

    double total = orderService
        .compute(100.0, "A01");
    assertEquals(80.0, total);
    verify(orderRepo).findTier("A01");
  }
}
```

Integración: Spring Boot + Base de Datos

```
@SpringBootTest(webEnvironment =
    WebEnvironment.RANDOM_PORT)
class OrderIntegrationTest {
  @Autowired
  private OrderRepository repository;

  @Test
  void shouldPersistAndRetrieveOrder() {
    Order order = new Order("ORD-1", 100.0);
    repository.save(order);

    Optional<Order> found =
        repository.findById("ORD-1");
    assertTrue(found.isPresent());
    assertEquals(100.0,
        found.get().getAmount());
  }
}
```

Ruta de Aprendizaje de la Sesión

1. Estrategias de Branching y Gobernanza Git
2. Pirámide y Automatización de Pruebas en el Build
3. **Cobertura con JaCoCo y Análisis Estático con SonarQube**
4. Orquestación CI/CD: Jenkins, Quality Gates y Nexus

Universidad icesi

03

Métricas de Calidad y SonarQube

JaCoCo, Deuda Técnica, Seguridad SAST y la filosofía Clean as You Code

Medición de Cobertura de Código con JaCoCo

Instrumentación de bytecode durante la ejecución de los tests:

- **¿Cómo opera?:** `jacoco-maven-plugin` inyecta sondas en los binarios compilados para registrar qué líneas y bifurcaciones se ejecutan.
- **Line Coverage:** Porcentaje de sentencias ejecutadas por la suite de pruebas.
- **Branch Coverage:** Porcentaje de caminos lógicos evaluados en estructuras condicionales (`if`, `else`, `switch`). Métrica mucho más rigurosa que la cobertura de líneas.
- **Artefactos generados:** Archivo binario de ejecución `jacoco.exec` y reporte estructurado en XML (`jacoco.xml`) para ingesta en SonarQube.

El Mito del 100 % de Cobertura

Un código con 100 % de cobertura puede tener cero aserciones válidas (*Assertion-free tests*) y estar plagado de bugs. La cobertura indica qué código **no** se probó, no demuestra que lo probado esté correcto.

SonarQube: Radiografía de la Calidad del Software

Pilares del Análisis Estático (SAST)

- **Bugs (Fiabilidad):** Defectos funcionales inminentes (`NullPointerException`, recursos sin cerrar).
- **Vulnerabilidades (Seguridad):** Brechas explotables en producción (Inyecciones SQL, credenciales expuestas, fallas OWASP).
- **Code Smells (Mantenibilidad):** Malos olores de diseño que elevan deuda técnica y complejidad ciclomática.
- **Duplicación de código:** Bloques repetidos que multiplican el costo de cambio.

Métricas y Calificaciones (Ratings)

- **Reliability Rating:** Letra A solo con 0 bugs abiertos.
- **Security Rating:** Letra A solo con 0 vulnerabilidades.
- **Security Hotspots:** Código sensible que requiere revisión arquitectónica manual.
- **Deuda Técnica (Technical Debt):** Tiempo estimado requerido para subsanar los code smells.

Filosofía Çlean as You Codez Quality Gates

Clean as You Code (New Code Focus)

La estrategia sostenible de saneamiento:

- Arreglar toda la deuda de un proyecto legado en bloque es inviable y paraliza el negocio.
- **El principio del boy scout:** El foco absoluto de SonarQube está en el **Código Nuevo** (*New Code*) introducido en la PR o versión reciente.
- Si cada línea nueva cumple el estándar de calidad, la base de código mejora progresivamente sin frenar la entrega de valor.

El Quality Gate: Filtro Binario

Criterios típicos de aprobación en New Code:

- **0 Nuevos Bugs** y **0 Nuevas Vulnerabilidades**.
- **Security Rating = A** y **Reliability = A**.
- **Cobertura en código nuevo** $\geq$ **80 %** (evaluada con el reporte JaCoCo).
- **Duplicación en código nuevo** $<$ **3 %**.
- **100 % de Security Hotspots revisados**.

Resultado estricto: PASSED (Verde) o FAILED (Rojo).

Ruta de Aprendizaje de la Sesión

1. Estrategias de Branching y Gobernanza Git
2. Pirámide y Automatización de Pruebas en el Build
3. Cobertura con JaCoCo y Análisis Estático con SonarQube
4. **Orquestación CI/CD: Jenkins, Quality Gates y Nexus**

Universidad icesi

04

Orquestación CI/CD con Jenkins y Nexus

Cerrando el Ciclo: De la PR al Artefacto Inmutable Aprobado

Conectando los Puntos: Jenkins, Calidad y Nexus

Jenkins: El Orquestador de la Cadena

- En sesiones anteriores vimos la arquitectura Controller–Agent, pipelines declarativos y webhooks con Smee.io.
- Jenkins no evalúa calidad por sí mismo: **invoca a Maven y JaCoCo**, despacha el escaneo a SonarQube y actúa como árbitro del veredicto.
- Notifica a GitHub/Bitbucket para habilitar o bloquear el botón de merge de la Pull Request.

Nexus: El Repositorio Inmutable

- En la sesión de empaquetamiento aprendimos que Nexus almacena binarios inmutables (`.jar`, Docker images).
- **Regla de oro de DevOps:** Un artefacto **NUNCA** se publica en Nexus si las pruebas fallaron o si el Quality Gate de SonarQube fue rechazado.
- Nexus solo aloja artefactos cuya trazabilidad y calidad fueron certificadas por el pipeline de CI.

Jenkinsfile: Pruebas, Cobertura, SonarQube y Nexus

Pipeline CI: Pruebas & SonarQube

```
pipeline {
  agent {
    docker { image 'maven:3.9-temurin-17' }
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Unit & IT Tests') {
      steps {
        // Ejecuta pruebas y JaCoCo
        sh 'mvn clean verify jacoco:report'
      }
    }
    stage('SonarQube Static Analysis') {
      steps {
        withSonarQubeEnv('SonarQube-Server') {
          sh 'mvn sonar:sonar \
           -Dsonar.coverage.jacoco.xmlReportPaths=\
            target/site/jacoco/jacoco.xml'
        }
      }
    }
```

Gobernanza: Quality Gate & Nexus

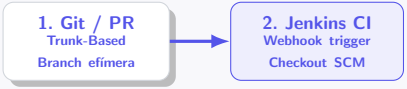
```
    stage('Quality Gate Enforcement') {
      steps {
        timeout(time: 2, unit: 'MINUTES') {
          script {
            def qg = waitForQualityGate()
            if (qg.status != 'OK') {
              error "QG Rechazado: ${qg.status}"
            }
          }
        }
      }
    }
    stage('Publish to Nexus') {
      when { branch 'main' }
      steps {
        // Solo publica si QG aprobo
        sh 'mvn deploy -DskipTests'
      }
    }
  }
}
```

Flujo Integral de Gobernanza y Calidad en CI/CD

1. Git / PR
Trunk-Based
Branch efímera

Paso 1: Desarrollador crea PR en rama efímera siguiendo Trunk-Based Development

Flujo Integral de Gobernanza y Calidad en CI/CD



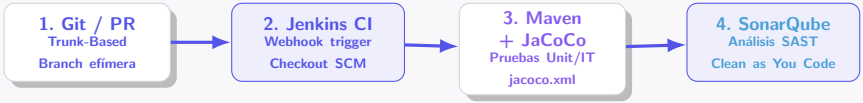
Paso 2: Webhook notifica a Jenkins CI y dispara ejecución en contenedor aislado

Flujo Integral de Gobernanza y Calidad en CI/CD



Paso 3: Maven ejecuta pruebas unitarias y JaCoCo registra cobertura de líneas y ramas

Flujo Integral de Gobernanza y Calidad en CI/CD



Paso 4: SonarQube realiza análisis SAST de bugs, vulnerabilidades y code smells

Flujo Integral de Gobernanza y Calidad en CI/CD



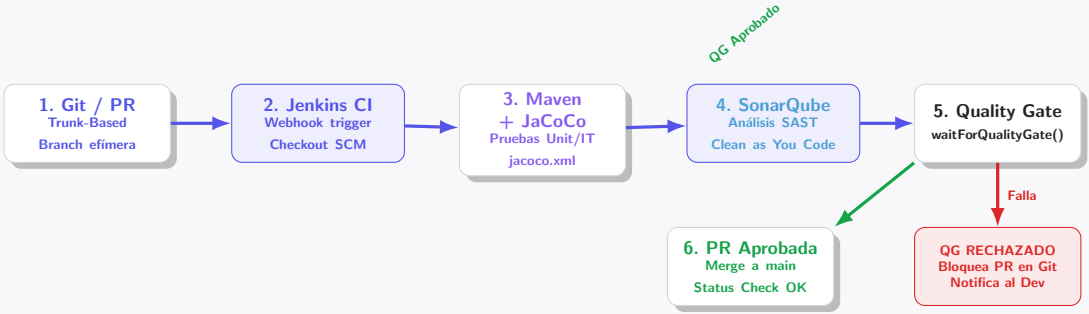
Paso 5: Jenkins pausa con waitForQualityGate() a la espera del Webhook de SonarQube

Flujo Integral de Gobernanza y Calidad en CI/CD



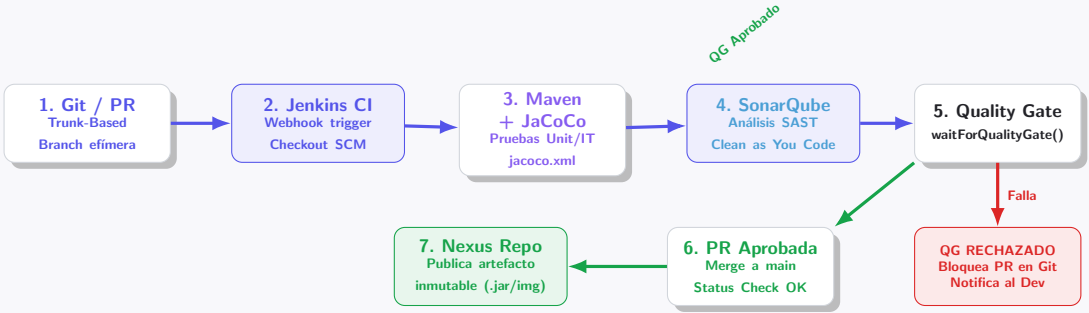
Bifurcación: QG no superado → Pipeline falla, PR bloqueada y feedback al desarrollador

Flujo Integral de Gobernanza y Calidad en CI/CD



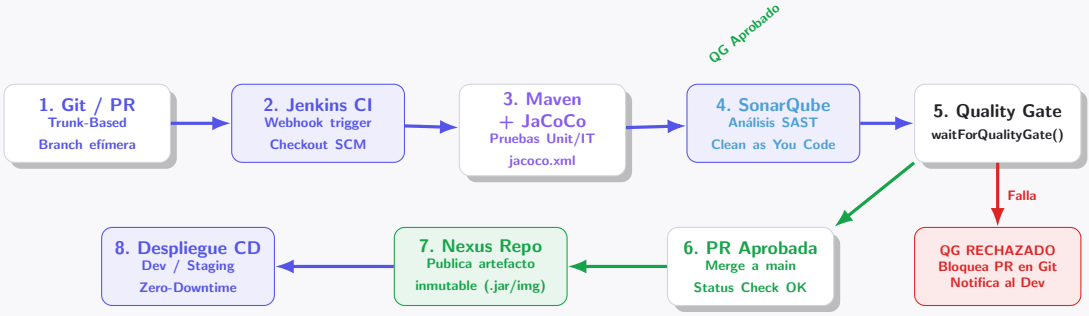
Bifurcación: QG superado → Status Check OK en GitHub/Git y merge habilitado a main

Flujo Integral de Gobernanza y Calidad en CI/CD



Paso 7: Publicación del binario e imagen inmutable en Sonatype Nexus OSS

Flujo Integral de Gobernanza y Calidad en CI/CD



Paso 8: Despliegue Continuo (CD) y validación automatizada de disponibilidad HTTP

Shift-Left

El Costo Exponencial de un Defecto

"Detectar un error en el entorno local cuesta 1x; capturarlo en el pipeline de CI cuesta 10x; descubrirlo en producción cuesta 100x y la reputación operativa del sistema."

Pregunta de Reflexión 1: Branching a Escala

Pregunta 1: Trunk-Based vs. GitFlow en Escala

¿Por qué gigantes tecnológicos como Google, Meta y Netflix abandonaron GitFlow a favor de Trunk-Based Development con Feature Flags?

Pregunta de Reflexión 1: Branching a Escala

Pregunta 1: Trunk-Based vs. GitFlow en Escala

¿Por qué gigantes tecnológicos como Google, Meta y Netflix abandonaron GitFlow a favor de Trunk-Based Development con Feature Flags?

Análisis y Respuesta de Ingeniería

- **Eliminación del Merge Hell:** Las ramas de larga duración provocan colisiones masivas y retrasan la integración semanas. En TBD, la integración es diaria y atómica.
- **Desacople de Despliegue y Liberación:** Con *Feature Flags*, el código se despliega a producción continuamente pero se activa dinámicamente sin bifurcar ramas de release.
- **Feedback Rápido en CI:** Cada commit ejecuta la suite completa sobre la verdad única (`main`), reduciendo el tiempo de detección de fallos a minutos.

Pregunta de Reflexión 2: Tiempo de Feedback en CI

Pregunta 2: El Dilema del Tiempo en Pipelines de CI

Si una suite de pruebas de integración tarda 45 minutos en completarse, ¿cómo balancear la velocidad del feedback en la PR sin sacrificar cobertura?

Pregunta de Reflexión 2: Tiempo de Feedback en CI

Pregunta 2: El Dilema del Tiempo en Pipelines de CI

Si una suite de pruebas de integración tarda 45 minutos en completarse, ¿cómo balancear la velocidad del feedback en la PR sin sacrificar cobertura?

Análisis y Respuesta de Ingeniería

- **Estratificación de Pipelines (Test Splitting):** En la PR solo corren pruebas unitarias y smoke tests críticos (< 5 min) para habilitar feedback ágil y merge fluido.
- **Paralelización en Contenedores:** Distribuir la suite de integración en múltiples agentes/workers concurrentes de Jenkins usando Testcontainers.
- **Ejecución Nocturna / Post-Merge:** Pruebas de integración pesadas, regresión completa y carga corren asíncronamente post-merge o en *Nightly Builds*.

Pregunta de Reflexión 3: Calidad vs. Métricas Ciegas

Pregunta 3: La Trampa de las Métricas de Cobertura

¿Qué incentivos perversos puede provocar en los desarrolladores establecer un objetivo corporativo ciego del "95 % de cobertura en JaCoCo"?

Pregunta de Reflexión 3: Calidad vs. Métricas Ciegas

Pregunta 3: La Trampa de las Métricas de Cobertura

¿Qué incentivos perversos puede provocar en los desarrolladores establecer un objetivo corporativo ciego del "95 % de cobertura en JaCoCo"?

Análisis y Respuesta de Ingeniería

- **Ley de Goodhart:** Al convertir la métrica en meta, se crean pruebas vacías que ejecutan líneas sin validar aserciones reales (*assertion-free tests*).
- **Falsa Sensación de Seguridad:** Se sobre-prueban getters, setters y DTOs inflando la estadística, descuidando caminos de error y bifurcaciones complejas.
- **Solución en Ingeniería:** Exigir *Branch Coverage* en lógica de negocio crítica, junto a análisis estático en SonarQube y pruebas de mutación (Pitest).

Pregunta de Reflexión 4: Gestión de Incidentes Críticos

Pregunta 4: Bypass de Quality Gate en Incidentes P1

Durante una caída crítica en producción un fin de semana, ¿debería permitirse eludir (*bypass*) el Quality Gate para desplegar un hotfix?

Pregunta de Reflexión 4: Gestión de Incidentes Críticos

Pregunta 4: Bypass de Quality Gate en Incidentes P1

Durante una caída crítica en producción un fin de semana, ¿debería permitirse eludir (*bypass*) el Quality Gate para desplegar un hotfix?

Análisis y Respuesta de Ingeniería

- **Riesgo de Agravamiento Mayor:** Eludir compuertas de calidad suele generar caídas secundarias más destructivas por defectos no verificados en el parche.
- **Rollback antes que Hotfix a Ciegas:** La mejor práctica es revertir de inmediato al artefacto inmutable anterior en Nexus o apagar el *feature flag*.
- **Excepción Estrictamente Auditada:** Si el parche es imperativo, requiere aprobación dual (Lead DevOps / CTO), ticket P1 y remediación prioritaria en el sprint.

Pregunta de Reflexión 5: Inmutabilidad y Gobernanza

Pregunta 5: Nexus y la Inmutabilidad de Artefactos

¿Por qué desplegar un artefacto generado manualmente en la laptop de un desarrollador directamente a Nexus viola los principios DevOps?

Pregunta de Reflexión 5: Inmutabilidad y Gobernanza

Pregunta 5: Nexus y la Inmutabilidad de Artefactos

¿Por qué desplegar un artefacto generado manualmente en la laptop de un desarrollador directamente a Nexus viola los principios DevOps?

Análisis y Respuesta de Ingeniería

- **Riesgo "En mi máquina funciona":** El binario local puede compilarse con JDK o dependencias dispares, variables residuales o cambios locales no versionados.
- **Ruptura de Cadena de Custodia:** Se pierde la trazabilidad del commit SHA exacto y se evaden los controles de seguridad SAST y análisis de calidad.
- **Principio de Única Fuente de Verdad:** Solo los agentes de Jenkins en entornos aislados y limpios tienen credenciales para firmar y publicar en Nexus.